Mujhe ek aisa app banana hai bilkul flipaclip jaisa, log users animators haath se canvas par draw karke animations bana sake, usme color de sake, kuch bhi kar sake, ek color color picker se color de sake, users apne haath se canvas par kuch bhi apne haath se draw karke animation bana sake. Lekin animation keyframe se hoga, ek white canvas aur uske niche timeline keyframes add karne ke liye, dono types se animation hogi user select kar sakta hai ki use frame by frame animation karna hai ya keyframe based.
color -
User draw kiye hue chij me color de sakta hai, color ki opacity, change kar sakta hai, hue de sakta hai, sabkuch change kar sakta hai.

ab iss app me bohot sare features honge.
mostly sabse powerful frame by frame animation hoga. jaise user ko white canvas diya jayega, uspar draw karega, jab user niche + par click karega to dusra frame aa jayega.
Isme Bohot sare tools honge, user jitna bhi usne draw Kiya hai sabko eksaath ek click me copy kar sakta hai aur agle ya kisi bhi frame me paste kar sakta hai, jis frame me paste Kiya hai vo uss frame me bhi dikhega.

matlab bilkul flipaclip jaisa sabhi features aur animation logic exactly same, rahega, pehle deep research Karo aur samjho ki kaise flipaclip animation banta hai. Isme aisa features hoga flipaclip jaisa ki user animator draw kiye hue jo bhi kuch hai use select karke canvas par kahi bhi move kar sakta hai hai real time me. jaise user ne select tool se draw kiye hue part ko select Kiya, aur use vo kahi bhi move kar sakta hai. Lekin ek Bohot powerful feature hoga ise saath. Ek user ek Animator agar ek caracter banata hai to draw karke vo har body parts ko alag alag banayega, aisa rule nahi ki alag alag banayega lekin ek soch hai jaise same canvas par thodi dur dur body parts banayega ye animator par depend karta hai. Ye uske haath me ye hai koi app ka rule nahi hai lekin mai tumhe batane ke liye keh raha hun for example kisi ek animator ne ek white canvas frame par ek caracter banaya sabhi body parts alag banaye. For vo har body parts ko ab caracter ke saath jod deta hai, jisse ek compete caracter bana hua dikhe,
ab sabse powerful features hoga, pivot points, single or multiple. User select kar sakta hai ki vo single pivot point lagana chahta hai ya multiple. Vo jitne chahe utne pivot points kisi ek jagah par ya multiple jagah par laga sakta hai, jisse jaha pivot point lage vaha se vo chij detach na ho sake, aur vaha se hil nahi sake, matlab vaha se nahi hilegi lekin dusri jagah se hilegi, uss kisi ek draw ki gayi chij ko kisi bhi frame par select karke hila sakte hai.
flipaclip me kya hota hai ki ham point to laga dete hai uss chij par lekin jab ham puri draw ki hui chij ko copy karte hai aur agle frame par paste karte hai to agle frame par uss chij ke kisi ek chij ko firse select nahi kar sakte hilane ke liye. Isliye problem hoti hai.
User iss app me pure ke pure scene ko bhi copy paste kar sake.
jab kisi draw kiye hue part ko ham select kare to uspar ek rectangle ya square aa jaye jisse ham uss chij ko hila sake, vo chij sirf do direction me ghum sakti hai x aur y. Jab user ise y direct me haath se khinche to ya to usko height badhegi. Dekho aur ke feature, user ne jo bhi chij draw ki hai, vo usko select karke tools se usko height width change kar sakta hai, user iss app me aur ek option hoga, group, parent child relationship, jaise agar user koi caracter bana raha hai to vo iska istemal kar sakta hai, jaise body parent aur koi aur chij child. ye sirf caracter ke liye nahi har chij ke liye hoga ye user par depend karta hai ki kya vo ise select karta hai ya nahi. kisi bhi chij ke liye. matlab pehle user draw karta hai, fir left manu se select karta hai ki ye parent hoga ya child aur aage parent hai to agli jo chij vo draw karega, usko select karke menu me ye select karega ki move to jo chij usne abi uss user ne

phele banayi thi. Fir vo chij pehli chij ke under chali jayegi. Jisse un dono me se koi bhi chij select ho dono chij eksaath hilegi. aage piche upar niche hogi. aur jab user iss chij me se kisi ek individual parts ya chij ki height ya width badhayega to y direction me height badhegi x direction me width. for example use ne ek caracter create Kiya, head, torso body legs arms sabkuch, aur pivot points lagakar unhe jod diya. Pehle frame par caracter bana hai, ab vo + icon par click karke agla frame banayega, vo frame white blank hoga uspar kuch nahi hoga, jab user sabko select karega. Aur copy karega aur angle frame par paste karega to vaisi ki vaisi drawing ya caracter agle frame paste hogi, ab same drawing dono frame par hai. agar vo sirf koi individual par ko select karke copy karta to sirf vahi chij agle frame par paste hoti puri body nahi, ab caracter agle frame par bhi hai, ab ise walk cycle create karna hai to vo pure caracter ko eksaath select karta hai aur thoda aage badhata hai, .second frame par pair ko select karta hai, aur aage rotate move karta hai to pair upar uthta hai, dusra pair piche karta hai isi tarah dono haath karta hai. abhi vo third frame lagata hai, uspar bhi opposite direction me lagata hai, aise hi bohot sare frames create karta hai aur har ek frames ke opposite direction me lagata hai, aur fir play button dabaya hai, har frame frame by frame play hota hai, ye koi video nahi frames hi hai fast play hone par ye video lagta hai. iss app me layers ka options hoga, multiple layers par multiple drawings hogi, agar background first layer par hai to caracter aur animation second layer par jaise background piche aur animation aage dikhega.

Select individual ya select all ka option hoga, kisi chij ko lock karne ka option jisse vo chij select nahi hogi ya uspar koi operation perform nahi hogi, jisse random achank usme koi galti se change nahi hoga.

Iss app me ek aur feature hoga, isme lines hogi jo user laga sakta hai, canvas par Animation ke liye jab ham blank canvas par Animation karte hai to ek frame me thoda upar agle frame me thoda niche ho jata hai, isliye ekbaar lines lagane par vo har frame me dikhegi straight lines jinhe drag kiya ja sakte hai kahi bhi. ek baar line lagane par animation uss line ke niche nahi ja sakti ya koi bhi drawing usse niche nahi ja sakti hai, lines sirf us drawing kisi ek chij ke liye hogi jisko select karke lines lagayi gayi hai. Varna jitni bhi drawings hai sabhi uske andar hi fir ho jayegi.
For example user ne pure caracter ko select Kiya fir lines lagayi, lines sirf caracter ke liye hogi ye ensure karne ke liye ki caracter uss lines ke niche na jaye. sirf caracter. Jo chij select thi usi ke liye. aur do lines hogi ek upar ek niche., jinhe drag kiya ja sakta hai, individual ya dono ek saath, matlab dono upar niche ho sakti hai, matlab dur ya pass.
isme baki sabhi features tum khud socho. Aur implement karo apne aap. light theme hona chahiye.
Onion Skinning: Ye animators ke liye sabse zaruri hai. Piche wale frame ki drawing ko thoda transparent (ghost like) dikhana, taaki pata chale ki agle frame me kitna aage badhana hai.
Lasso Select Tool: Ek freehand selection tool jisse user aas-paas gol ghera banakar multiple cheezon ko ek sath select kar sake.
Playback Controls: Loop, Play, Pause, aur FPS (Frames per second) control karne ka option. 12fps aur 24fps standard hote hain.
Export to MP4/GIF: Pura animation banne ke baad usko render karke save karne ka option.
Dhyan rakho pivot points. Aur selection most important hai, isi liye ye app bana raha hun, sabhi tools ke naam bhi dikhne chahiye. direct naam jaise selection, pivot. Etc. ya fir har tool ka first letter ya three letter, jaise pivot ke liye pvt slt, jab user pivot par click kare to canvas par ek goal pink ya red point aa jana chahiye, jisko user drag karke kahi bhi laga sakta hai,

ya use delete kar sakta hai, sabse main features select hai, matlab koi chij ya drawing kisi bhi frame par ho, uss drawing apr click karte hi vo select ho jana chahiye, sirf vahi jispar click Kiya hai, aur left side me bhi use select kar sakte hai, left ya right side me dikhega ki kon kon si individual drawing hai ya unhe ham naam de sakte hai, individual, by default, untitled1 untitled2 3 aise hoga use edit kar sakte ya to canvas par click karke select kare ya fir manu se. yaad rakho pivot aur selection must hai. Aur groups bhi. animation hona chahiye. bohot sare tools add karo. Brush aur sabkuch. color picker ya hex code se bhi color de sake. mix color bhi de sakte hai.
Touch se drawing select kar sakte hai dobara touch karne par vo election gayab ho jayega. Aisa hi hoga jab ekbaar select hoga tab drawing ke charo taraf rectangle controls aayega firse tap karke par vo gayab ho jayega. jab pehle Wale frame par draw Kiya ab dusra frame add kiya to pehle Wale frame ki opacity bohot kam dikhni chahiye. Taki agle frame ki drawing kaha se ye dikh sake aur clear drawing dikh sake.

Main features -
Touch or tap on drawing to select Automatically and apply any operation.
pivot points. Color picker appy in real time.
sabse jaruri hai ki jab user koi drawing banaye to use select kar sake, drawing par tap karte hai vo select ho jayegi, sabhi operation ussi drawing ke liye perform honge jisko select Kiya Gaya hai aur, jab firste tap karke use unselect kiya tab operation perform nahi hogi. Jab drawing ko user select kare tabhi sabhi options aane chahiye.

Aur e features -

App me sabhi prakar ke shapes honge, circle, rectangle, triangle. Sabkuch, jisse select karke user drawing kar sakta hai. un shapes ko modify kar sakta hai. Shapes se vo kuch bhi bana sakta hai. sabse jaruri par pivot points honge.

User kisi bhi chij ke liye child parent relationship ki tarah drawing ko bana sakta hai. Jaise for example user ne koi drawing banayi vo drawing left ya right panel me dikhegi, jab vo multiple drawings banayega tab, right ya left panel me dikhegi sabhi drawings aur jab vo create groups par click karega tab left panel ko select ya drag karke kisi ko kisi ka child ya parent bana sakta hai, ek baar banane ke baad drawing parent child ko tarah dikhegi, jaise file structure hota hai na vaise jaise ek folder uske niche koi dusra folder ya file aise hi unlimeted nested child parent system, isse jab uss group ki koi chij drag hogi to usse sabhi group ke members drag hokar uske saath drag hone ya hilenge. group ke kisi ek individual parts ya drawing child ke saath operation perform hoga to vo sirf usi par lagu hogi jsie select karke operation perform kiya gaya hai, for example color ya pivot point. ya fir rotate ya scene karna ya koi bhi dusra lekin jab x  ya fir x directorate me user use apne haath se move karega tab sabhi groups ke members move honge.
bones system ki taraf lekin sirf caracter ke liye nahi sabhi chijo ke liye. Sabhi drawings ke liye.

Aur bhi features suggest Karo jo time save kare aur jaldi achhe result lakar de, bohot achhi animation bankar taiyaar ho bohot kam samay me, anya animation softwares se kuch tools copy karo aur kuch apne hisaab se sochkar khud banao,  jo kisi anya animation app me na ho, isliye to ye app banaya ja raha hai taki animation jaldi ho sake. aur detailed me batao. khud se soche hue new 10 + features jo free aur bina internet ke chal sake, yani sirf logic se

controls. Sirf logic lagane ko jarurat hai, chijo ko sochkar combine karke banaya ja sakta hai, jaise flipaclip ek bohot achha app hai lekin usme kich features nahi hai jaise pivot points, aur shapes ka.

Problem sirf select ka hai, dekho, jab pehle frame par koi chij draw ki, aur ab use hame agle frame par le jana hai, use copy karke agla frame lagakar paste kar denge lekin pehle frame me vo select hokar uska controls area aata tha jisse use manuplate kiya ja sake, lekin second frame me select ka option nahi aata, for example dusre frame par agar ussi drawing ko tap karke select Kiya to sabhi options aane chahiye, jaise move rotate sabkuch. isse kya hoga ki, dusre frame par Animation ke liye uss same chij ke liye alag se animation ke liye firse draw karne ki jarurat nahi hogi, direct select, move or rotate any perform. aur fir agla frame vaha bhi kuch operation, aise hi ek Animation ban jayegi sirf kisi ek individual chij ko ekbaar draw karke ki jarurat hogi, isse har frame me use select karke par move rotate scale, height width, sabhi options show honge. Jitne bhi frame banege utne me sabhi tools available honge.
Puri drawing ka area select nahi hona chahiye sirf drawings, matlab agar drawing white canvas par bani hai aur ek gol circle hai to sirf uske aaju baju ki lines ya sirf circle select hoga, use niche ka ya uske aas paas ka white area nahi sirf lines. Aur sabhi features aur options sirf lines aani actual drawing ke saath perform honge, chahe copy paste ya kuch aur ho.

Tum jo app describe kar rahe ho, wo sirf Flipaclip clone nahi hai. Agar sahi design kiya jaye to ye Frame-by-Frame + Vector Rigging + Bone-less Animation + Scene Composition ka hybrid software ban sakta hai.

Main pehle architecture samjhata hoon.

Core Concept

Har drawing ko pixels ki tarah save nahi karna chahiye.

Har brush stroke, shape, object ko alag entity banana chahiye.

Example:

Scene
├── Layer 1 (Background)
│ ├── Tree
│ ├── Sky
│
├── Layer 2 (Character)
├── Head
├── Body
├── Left Arm
├── Right Arm
├── Left Leg
├── Right Leg

Isliye user kisi bhi drawing par tap kare aur vo select ho jaye.

---

Drawing Engine

Har drawing object:

```
{
 id:"obj_1",
 name:"Head",
 type:"brush",
 points:[...],
 color:"#ff0000",
 opacity:1,
 x:100,
 y:200,
 rotation:0,
 scaleX:1,
 scaleY:1,
 pivots:[]
}
```

Yahi sabse important hai.

Flipaclip ki tarah canvas flatten nahi karna.

Har object independent hona chahiye.

Tabhi selection possible hoga.

---

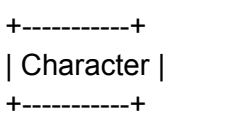
Selection System

Sabse important feature.

User touch kare:

Tap Object
↓
Hit Detection
↓
Object Selected
↓
Bounding Box Show

Bounding Box:




Handles:

↖ ↑ ↗

← →

↙ ↓ ↘

Use:

Move

Rotate

Scale

Stretch

---

Multi Selection

Tap = Single Select

Double Tap = Group Select

Lasso = Free Select

Select All

---

Persistent Selection

Flipaclip ki sabse badi problem:

Copy paste ke baad object identity khatam.

Tumhare app me:

Head
ID:123

Frame1
Frame2
Frame3
Frame4

Har frame me wahi ID maintain.

Isliye future frame me bhi select ho sakta hai.

---

Pivot System (Most Important)

Tumhara unique feature.

Object:

Arm

Pivot:

●
|
|
|

User:

PVT Tool

Select kare.

Red point:

🔴

Canvas par aa jaye.

Drag karke kahi bhi.

---

Multiple Pivot Points

Example:

Finger

●──●──●

Har joint alag.

Unlimited pivots.

---

Smart Pivot Lock

Tumhara special feature.

Pivot lagne ke baad:

Rotate

Object pivot se chipka rahega.

Detach nahi hoga.

---

Parent Child System

Example:

```
Body
 ├─ Head
 ├─ Arm L
 ├─ Arm R
 ├─ Leg L
 └─ Leg R
```

Hierarchy Panel:

```
Character
 ├ Head
 ├ Body
 ├ ArmL
```

├ ArmR
├ LegL
└ LegR

Drag & Drop.

Bilkul folder structure.

Unlimited nesting.

---

Smart Character Rig

User:

Select Head
Parent = Body

Done.

Ab:

Move Body

↓

Sab move.

Move Head

↓

Sirf Head.

---

Frame By Frame Timeline

[1][2][3][4][5]

Plus:

+

New frame.

---

Onion Skinning

Frame 1

Ghost 20%

Frame 2

Current

Frame 3

Ghost Ahead

User dekh sakta hai motion.

---

Keyframe Mode

Do modes:

Frame by Frame

OR

Keyframe Animation

Example:

Frame1

X=100

Frame20

X=500

Auto interpolate.

---

Layers

Layer 1 Sky

Layer 2 Mountains

Layer 3 Character

Layer 4 Effects

Drag reorder.

Lock.

Hide.

Opacity.

Blend Modes.

---

Constraint Lines (Tumhara Idea)

Character select.

Add Line.

---

Top.

---

Bottom.

Character:

Allowed Zone

Ke andar hi rahega.

Animation guide.

Bahut useful feature.

---

Shape Tools

Tools:

Rectangle

Circle

Triangle

Polygon

Star

Arrow

Line

Bezier

---

Color System

Features:

Color Picker

Hex

RGB

HSL

HSV

Opacity

Gradient

Palette Save

---

Brush Engine

Brushes:

Pencil

Pen

Marker

Airbrush

Calligraphy

Ink

Pixel

Texture

---

Playback

▸ Play

⏸ Pause

■ Stop

↺ Loop

FPS

FPS:

12
24
30
60

---

Export

Formats:

PNG Sequence

GIF

MP4

WebM

JSON Project

---

Auto Save

Every:

10 sec

Project save.

---

Additional Features (New Ideas)

Ye Flipaclip me nahi ya properly nahi hain.

1. Auto Inbetween Generator

Frame1

Hand down.

Frame10

Hand up.

App:

Generate Inbetween

Frames 2-9 create.

Offline.

Pure logic.

---

2. Motion Trail

Character ke piche:

• • • • •

Path dikhaye.

Animation smooth.

---

3. Smart Walk Cycle Maker

User:

Character Select

↓

Generate Walk Cycle

↓

Basic walk ready.

Animator edit kar sakta hai.

---

4. Pivot Templates

Human

Animal

Bird

Spider

Auto pivots.

---

5. Pose Library

Pose save:

Idle

Run

Jump

Attack

One click.

---

6. Symmetry Animation

Left arm move.

Right arm automatically.

---

7. Mirror Drawing

Canvas ke center par mirror.

Character drawing fast.

---

8. Motion Path Editor

Character:

Curve Path

Par chale.

---

9. Smart Duplicate

Character copy.

Animation bhi copy.

---

10. Global Color Replace

Red shirt.

Ek click:

Red → Blue

Saare frames update.

---

11. Character Asset System

Character save.

Dusre project me use.

---

12. Reusable Body Parts

Head Library
Hand Library
Eye Library
Hair Library

Drag & Drop.

---

13. Time Stretch

Walk cycle:

24 frames

↓

48 frames

Automatically slow.

---

14. Animation Comparison Mode

Screen split.

Old Animation
New Animation

Compare.

---

15. Infinite Undo History

Unlimited.

Project save ke baad bhi.

---

Sabse Powerful Feature

Main agar is app ka USP choose karun to:

Persistent Object Animation

Matlab:

Draw once

Har frame me selectable

Unlimited pivots

Parent-child hierarchy

Frame by frame

Keyframe

Onion skin

Ye combination Flipaclip, RoughAnimator aur ToonSquid ke beech ka hybrid ban jayega.

Agar ye sahi implement hua to mobile par professional 2D animation banana bahut fast ho sakta hai, especially tumhare persistent selection + multi-pivot + hierarchy system ki wajah se.

Haan, aur yahi tumhare app ka sabse powerful idea hai.

Tum asal me frame-by-frame drawing software nahi bana rahe, balki object-based animation software bana rahe ho.

Flipaclip me problem ye hai ki aksar frame ek bitmap image ki tarah treat hota hai. Jab tum frame copy karte ho, drawing dikh to jaati hai, lekin uske andar ke individual objects ki identity maintain nahi rehti.

Tumhare app me logic kuch aisa hona chahiye:

Galat Tarika

Frame 1:

Frame1
└── Image

Frame 2:

Frame2
└── Image Copy

Ab Frame2 me sirf image hai.

Tap karne par pata nahi chalega ki head kya tha, arm kya tha.

Isi liye selection mushkil ho jati hai.

---

Sahi Tarika

Frame1:

Head  ID=1
Body  ID=2
ArmL  ID=3
ArmR  ID=4
LegL  ID=5
LegR  ID=6

Frame2 me copy karte waqt:

Head  ID=1
Body  ID=2
ArmL  ID=3
ArmR  ID=4
LegL  ID=5
LegR  ID=6

IDs same rahengi.

Sirf transform alag hoga.

Example:

Frame1

ArmL

x=100
y=100
rotation=0

Frame2

ArmL

x=110
y=90
rotation=20

Object wahi hai.

Bas state badli hai.

---

Iska Result

Frame2 me:

Tap ArmL

↓

Select ho jayega.

↓

Bounding Box aayega.

↓

Rotate handle aayega.

↓

Scale handle aayega.

↓

Pivot points dikhenge.

↓

Parent-child relation dikhega.

Bilkul Frame1 ki tarah.

---

Aur Better Solution

Frame ko objects store hi nahi karne chahiye.

Frame ko sirf changes store karne chahiye.

Example:

Character:

Head
Body
ArmL

ArmR

Project me ek hi baar stored.

Frames:

Frame1
ArmL rotation=0

Frame2
ArmL rotation=20

Frame3
ArmL rotation=40

Frame4
ArmL rotation=60

Bas.

Isse:

Memory bahut kam lagegi

Selection har frame me kaam karega

Copy paste instant hoga

Undo fast hoga

Large projects smooth chalenge

---

Tumhara Desired Workflow

Animator:

Frame1

Head draw
Body draw
Arms draw
Legs draw

Character ready.

---

Frame2

Tap Leg

↓

Select.

↓

Rotate.

↓

Done.

---

Frame3

Tap Arm

↓

Select.

↓

Move.

↓

Done.

---

Frame4

Tap Head

↓

Select.

↓

Rotate.

↓

Done.

---

Pure animation me kabhi dubara draw karne ki zarurat nahi.

Sirf:

Select
Move
Rotate
Scale
Stretch

Yehi workflow rahega.

---

Aur agar tum pivot + parent-child bhi add kar dete ho to animator:

Body
 ├ Head
 ├ ArmL
 ├ ArmR
 ├ LegL
 └ LegR

ko ek baar draw karega.

Uske baad 100 frames ki walk cycle bhi bina redraw kiye bana sakta hai.

Mere hisab se tumhare app ka sabse important rule ye hona chahiye:

> "Jo object ek baar draw hua hai, wo project ke kisi bhi frame par hamesha selectable rahega."

Yahi rule tumhare app ko Flipaclip se fundamentally alag bana dega.

Bilkul sahi. Agar tumhara goal professional animation app banana hai, to selection canvas area based nahi, balki drawing geometry based honi chahiye.

Galat Selection

Agar user ne ek circle banaya:

```
+-----------+
|           |
|    ○      |
|           |
+-----------+
```

Aur poora rectangle select ho gaya, to problem hogi:

White empty area bhi select ho jayega.

Dusri drawings ke saath overlap issues.

Accurate animation mushkil.

Touch selection unreliable.

---

Sahi Selection

User ne circle draw kiya:

○

App ke liye object:

```
{
  id:1,
  points:[...circle path...]
}
```

Touch detection:

```
Finger Tap
   ↓
Path Hit Test
   ↓
```

Circle Selected

Sirf circle ki line/path select hogi.

---

Brush Drawing Ke Liye

User ne freehand draw kiya:

/\
/ \
\ /
\/

App ko pura canvas image save nahi karna.

Stroke save karna hai:

```
{
 id:5,
 type:"stroke",
 points:[
   {x:10,y:20},
   {x:15,y:25},
   {x:20,y:30}
 ]
}
```

Selection:

Tap stroke ke paas
↓
Stroke selected

White area select nahi hoga.

---

Fill Color Objects

Agar user ne circle ko fill kiya:

●

To selection ke 2 modes ho sakte hain:

Stroke Mode

Sirf outline select.

Shape Mode

Pura filled shape select.

Example:

Circle
 ├ Stroke
 └ Fill

Animator choose kar sakta hai.

---

Copy Paste

Copy karte waqt:

Circle Object

copy hoga.

Na ki:

Canvas Screenshot

Isliye paste ke baad bhi:

Select

Move

Rotate

Scale

Pivot

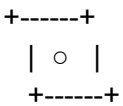
Parent Child

sab kaam karega.

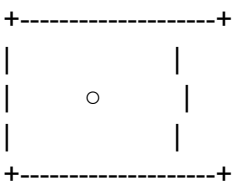
---

Bounding Box

Bounding box sirf drawing ke aas paas:



Na ki:

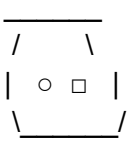


---

Lasso Selection

Lasso bhi geometry based hona chahiye.

User:



To app check kare:

Kaunsi drawings polygon ke andar hain?

Sirf wahi select hongi.

---

Smart Hit Detection

Touch screen ke liye:

Agar line 2px hai to exact line tap karna mushkil.

Isliye invisible touch radius:

Line = 2px

Touch Radius = 15px

User line ke paas tap kare to bhi select ho jaye.

Lekin actual selection fir bhi line/object ki hi hogi.

---

Tumhare App Ka Golden Rule

> Canvas par kabhi kuch select nahi hota.

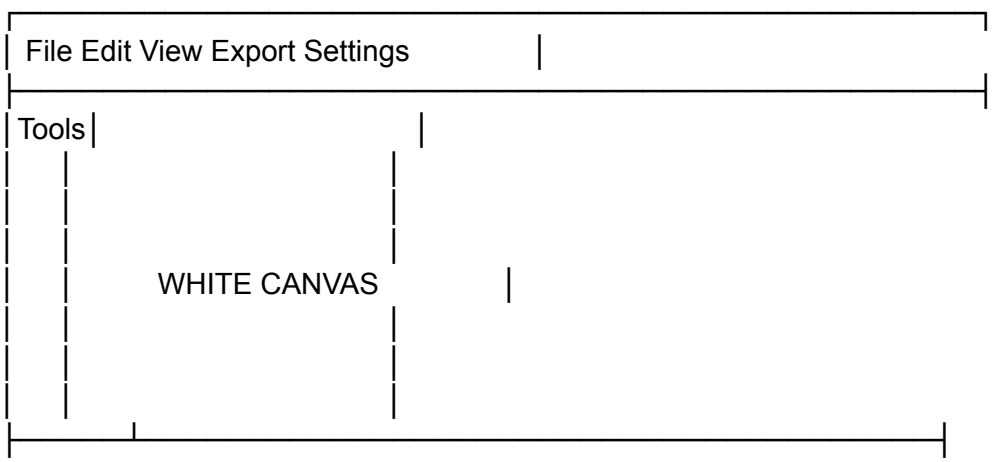
Sirf objects, strokes, shapes, groups, layers aur drawings select hoti hain.

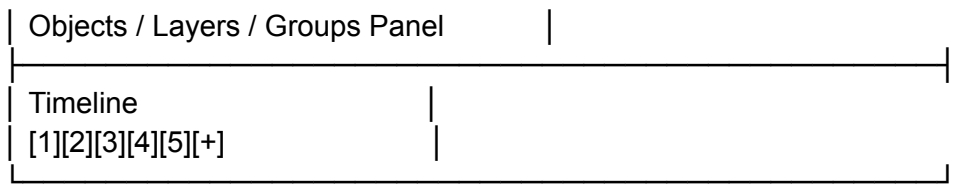
Yani agar canvas 5000×5000 bhi ho aur beech me ek chhota sa circle bana ho, to selection system ko sirf us circle ke vector/path/stroke ko hi detect karna chahiye. White background, khaali jagah aur canvas area kabhi selection ka part nahi honi chahiye. Isi wajah se copy-paste, pivot points, parent-child, move, rotate aur frame-to-frame animation sab perfectly kaam karenge.

Agar main tumhare poore idea ko professional animation software ki tarah design karun, to UI kuch is tarah hogi.

---

Main Screen Layout






Light theme.

Pure white canvas.

Light gray panels.

Blue selection highlights.

Red pivot points.

Green motion paths.

---

Left Toolbar

Vertical toolbar.

SEL
LAS
BRS
ERS
LIN
REC
CIR
TRI
POL
TXT
FIL
CLR
PVT
GRP
PAR
CHD
BON
MOT

---

SEL (Selection Tool)

Sabse important.

User tap:

Circle

↓

Instant select.

↓

Bounding box.

↖ ↑ ↗

← [○] →

↙ ↓ ↘

Visible.

---

Bounding Box Controls

Corner Drag:

Scale Uniform

Side Drag:

Width
Height

---

Top Handle

Rotate

---

Double Tap

Rename

---

Triple Tap

Object Settings

---

Object Settings Popup

Name

Opacity

Lock

Hide

Clone

Duplicate

Mirror

Color

Stroke Width

Blend Mode

Parent

Child

Pivot

---

Brush Tool

Tap BRS.

Top panel:

Brush
Pen
Marker
Pencil
Airbrush
Texture
Pixel
Calligraphy

---

Brush Settings

Size

Opacity

Flow

Hardness

Smoothing

Pressure

Spacing

Stabilizer

---

Real Time Preview

Brush stroke banne se pehle dikhai dega.

---

Shape Tools

Rectangle

Circle

Triangle

Polygon

Star

Arrow

Line

Bezier

Heart

Cloud

Speech Bubble

Gear

Custom Shape

---

Shape Draw

Drag.

Shape banegi.

---

Select Shape

Move.

Rotate.

Scale.

Color.

Pivot.

Group.

Sab kaam karega.

---

Color Panel

Open Color.

Large Color Wheel.

Hue
Saturation
Brightness
Opacity

---

Hex Input

#FF0000

---

RGB

255 0 0

---

HSL

0 100 50

---

Eyedropper

Canvas se color uthao.

---

Gradient

Linear
Radial
Angular

---

Palette

Unlimited save.

---

Frame Timeline

Bottom.

[1][2][3][4][5]

---

Frame Types

Normal

Keyframe

Hold Frame

Reference Frame

Locked Frame

---

Add Frame

+

Tap.

New frame.

---

Long Press Frame

Copy

Paste

Duplicate

Delete

Reverse

Mirror

Color Label

---

Onion Skin

Timeline par.

ONION

Button.

Tap.

---

Options

Previous Frames

Next Frames

Opacity

Color

---

Example

Frame 1 Red Ghost

Frame 2 Current

Frame 3 Blue Ghost

---

Object Panel

Right side.

Character

Tree

Ground

Sun

Cloud

---

Tap.

Select.

---

Long Press

Rename

Duplicate

Delete

Lock

Hide

---

Hierarchy System

Objects panel:

Character
 ├ Head
 ├ Body
 ├ Arm L
 ├ Arm R
 ├ Leg L
 └ Leg R

---

Drag Arm.

Drop into Body.

Parent Child.

---

Collapse Expand.

---

Unlimited nesting.

---

Pivot Tool

Tap PVT.

Canvas.

Red point.

🔴

---

Drag anywhere.

---

Pivot Settings

Single

Multiple

Locked

Free

Joint

Constraint

---

Multiple Pivot Example

Finger

●—●—●

---

Constraint System

Unique feature.

Select object.

Tap Constraint.

---

Canvas.

Upper Line
Lower Line

Appear.

---

Drag lines.

Object cannot leave zone.

---

Options

Vertical

Horizontal

Circle

Custom Shape

---

Motion Path

Select object.

Tap MOT.

---

Draw path.

~~~~~~~

---

Object follows.

---

Speed points.

Direction points.
~~~~~~~

---

Smart Walk Cycle

Select Character.

Tap:

AUTO WALK

---

Settings

Steps

Speed

Stride

Bounce

---

Generate.

---

Walk cycle ready.

---

Auto Inbetween

Frame1.

Arm Down.

Frame10.

Arm Up.

---

Tap.

Generate Inbetween

---

Frames 2-9 generated.

---

Pose Library

Save pose.

Idle

Run

Jump

Sit

Attack

---

Apply later.

---

Mirror Animation

Move Left Arm.

Right Arm automatically.

---

Settings

X Axis

Y Axis

Character Center

---

Layer Panel

Layer 1 Background

Layer 2 Character

Layer 3 Effects

---

Each Layer

Hide

Lock

Opacity

Blend Mode

---

Drag reorder.

---

Camera Layer

Special layer.

Move camera.

Zoom.

Rotate.

Pan.

---

Animation camera.

---

Export

GIF

MP4

WEBM

PNG Sequence

Project File

---

MP4 Settings

720p

1080p

1440p

4K

---

FPS

12

24

30

60

---

Advanced Feature 1

Live Pivot Recording

Pivot movement record.

Animation automatically.

---

Advanced Feature 2

Ghost Object Editing

Previous frame object directly drag.

Current frame update.

---

Advanced Feature 3

Smart Auto Naming

Untitled1

Untitled2

Untitled3

Automatically.

---

Advanced Feature 4

Animation Analyzer

Detect:

Jerky Motion

Uneven Spacing

Missing Frames

---

Advanced Feature 5

Timeline Heatmap

Red:

Heavy changes

Blue:

Few changes

---

Advanced Feature 6

Reusable Components

Save:

Hand

Eye

Mouth

Hair

Tree

Rock

---

Drag into any project.

---

Advanced Feature 7

Instant Character Builder

Draw parts.

Tap:

Build Character

Automatically creates:

hierarchy

pivots

groups

---

Advanced Feature 8

Frame Comparison

Split screen.

Frame 1 | Frame 50

Compare.

---

Advanced Feature 9

Infinite History

Every action stored.

Even after restart.

---

Advanced Feature 10

Project Asset Browser

Characters

Props

Backgrounds

Effects

Shapes

Saved Animations

Reusable forever.

---

Sabse important baat: tumhare app me selection, persistent objects, pivots, hierarchy aur frame-to-frame editable objects foundation hone chahiye. Baaki saare tools unke upar build honge. Agar ye foundation sahi bana, to animator ek baar character draw karke usko har frame me select, move, rotate, scale aur animate kar sakta hai bina dubara redraw kiye. Yahi tumhare app ka sabse bada difference hoga.

Yeh raha tumhare app ka **complete, frame-by-frame, click-to-click architecture aur UI/UX flow**. Maine isko ek professional animation software ki tarah design kiya hai jisme har button, har tap, aur har logic exactly define kiya gaya hai. Language Hindi + Technical English mix hai taaki developer aur designer dono samajh sakein.

---
## 📐 1. CORE ARCHITECTURE & DATA MODEL (Foundation)
Tumhara app **Object-Based Vector Animation Engine** hoga. Pixel/Bitmap nahi, har stroke/shape ek independent `Object Node` hoga.

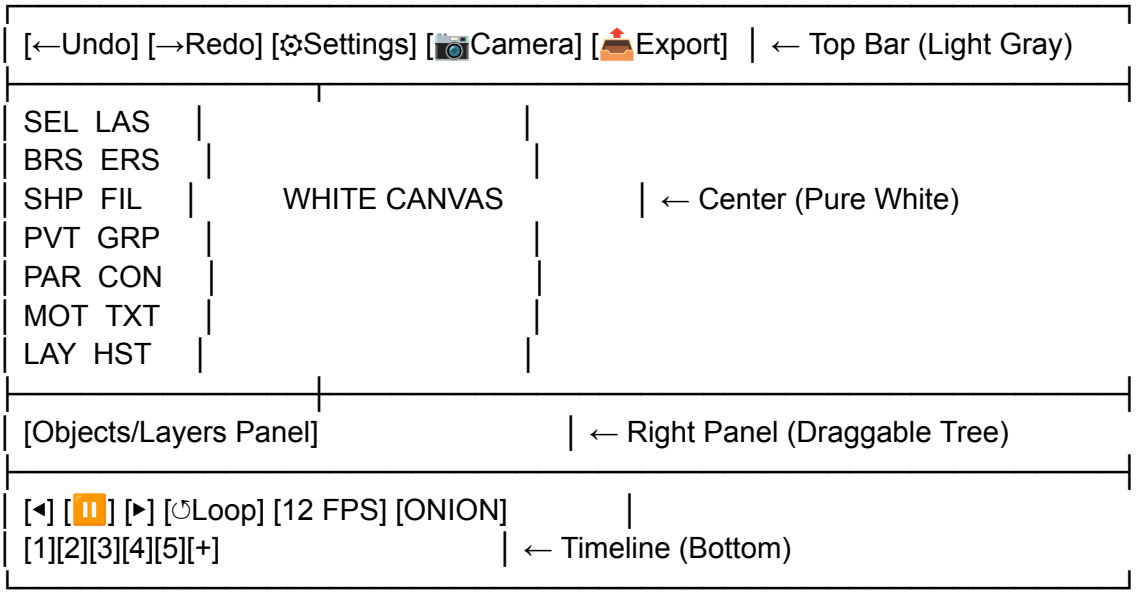
```json
{
  "project": {
    "id": "proj_01",
    "canvasSize": { "w": 1920, "h": 1080 },
    "fps": 12,
    "layers": [
      {
        "id": "layer_bg",
        "name": "Background",
        "visible": true,
        "locked": false,
        "opacity": 1.0,
```

```
      "blendMode": "normal",
      "frames": [
       {
        "index": 0,
        "objects": [
         {
          "id": "obj_tree_01",
          "type": "shape",
          "points": [[x1,y1],[x2,y2]...],
          "fill": "#2E7D32",
          "stroke": "#1B5E20",
          "strokeWidth": 2,
          "transform": { "x": 200, "y": 400, "scaleX": 1, "scaleY": 1, "rotation": 0 },
          "pivots": [ { "id": "pvt_1", "x": 0, "y": 0 } ],
          "parent": null,
          "constraints": { "type": "horizontal", "min": 100, "max": 600 }
         }
        ]
       }
      ]
     }
    ]
  }
}
```

**Golden Rule:** `Object ID` har frame me same rehta hai. Sirf `transform`, `pivots`, `color`, `constraints` change hote hain. Isliye copy-paste ke baad bhi selection, pivot, hierarchy kaam karti hai.

---

## 🖥️ 2. EXACT UI LAYOUT (Light Theme)



```
- **Light Theme:** `#FAFAFA` background, `#E0E0E0` panels, `#2196F3` selection, `#E53935` pivots, `#4CAF50` active tools.
- **Canvas:** Pinch-to-zoom, two-finger pan. Always white. Grid optional.

---
## 👆 3. CLICK / TAP FLOW (Step-by-Step Interaction)

### 🔹 SELECTION TOOL (SEL)
1. **Tap on drawing:** Engine `point-to-segment distance` calculate karta hai. Agar distance `< 12px` (touch radius), object selected.
2. **UI Response:** Blue bounding box + 8 handles (corners + midpoints) + top rotation handle.
3. **Drag corner:** Uniform scale.
4. **Drag side:** Width/Height independent stretch.
5. **Drag center:** Move.
6. **Drag top handle:** Rotate around pivot.
7. **Tap empty canvas:** Deselect. Bounding box gayab.
8. **Double tap object:** Rename popup (default `Untitled_1`).
9. **Long press object:** Context menu: `Lock, Hide, Duplicate, Delete, Settings`.

### 🔹 PIVOT TOOL (PVT)
1. **Tap PVT:** Canvas cursor changes to `🔴+`.
2. **Tap selected object:** Red pivot dot appears at center.
3. **Drag pivot:** Move freely. Rotation/scale ab iske around hoga.
4. **Shift+Tap or Panel+Add:** Multiple pivots add hote hain. Har joint par ek dot.
5. **Pivot Logic:**
   - `rotation = angle(current_point - pivot)`
   - `pivot_lock = true` → object detach nahi hoga pivot point se.
   - Right panel me `Pivots List` dikhti hai. Tap → edit position. Long press → delete.

### 🔹 PARENT-CHILD HIERARCHY (GRP / PAR / CHD)
1. **Right Panel:** Drag `Head` → drop on `Torso`. `Torso` ke under indent ho jata hai.
2. **UI:** Folder tree jaisa dikhega. `▶` expand/collapse.
3. **Logic:**
   - Parent move kare → child move.
   - Parent rotate → child rotate + pivot maintain.
   - Child independently move/rotate → parent unaffected.
   - `Transform Matrix Inheritance` use hoga. (Web standard: `child.globalTransform = parent.globalTransform × child.localTransform`)

### 🔹 CONSTRAINT LINES (CON)
1. Select object → Tap `CON`.
2. Canvas par 2 lines appear: `Top Line` (Blue), `Bottom Line` (Green).
3. Drag lines → adjust bounds.
4. Logic: `object.y = clamp(object.y, top_line.y, bottom_line.y)`
5. Object ko lines ke bahar drag karne par `elastic bounce + red flash` feedback.
6. Constraint sirf selected object/group ke liye active.
```

### 🔹 FRAMES & TIMELINE
1. **Tap `[+]`:** New frame create. Onion skin auto ON.
2. **Long press frame:** `Copy, Paste, Duplicate, Delete, Reverse, Mirror, Color Label`.
3. **Copy/Paste:** Sirf object data copy hota hai. Canvas screenshot nahi. ID same rehta hai.
4. **Frame types:** `Normal`, `Keyframe (Diamond Icon)`, `Hold (Square)`, `Onion Ref (Ghost)`.
5. **Playback:** `▶ Play` → frames render at FPS. `↺ Loop` toggle. `FPS Slider` 6-60.

### 🔹 ONION SKINNING
1. **Tap `ONION`:** Toggle ON/OFF.
2. **Settings Popup:**
   - `Previous Frames`: 1-5
   - `Next Frames`: 1-5
   - `Opacity`: 10%-50%
   - `Prev Color`: Red tint, `Next Color`: Blue tint
3. Render logic: Previous/Next objects ko `globalAlpha` ke saath draw karo. Current frame full opacity.

---
## ⚙️ 4. CORE SYSTEMS DEEP DIVE

### 🔹 Persistent Selection & Hit Detection
- Canvas ko `spatial hash grid` me divide karo (cell size = 64px).
- Har stroke/shape apne cells register karta hai.
- Tap hone par sirf usi cell ke objects check hote hain → `O(1)` performance.
- `point-to-polyline distance` algorithm se sirf line select hoti hai, white space ignore.

### 🔹 Pivot & Rotation Math
- Single Pivot: `x' = cx + (x-cx)cosθ - (y-cy)sinθ`
- Multiple Pivot: Joint chain me `forward kinematics` jaisa logic. Har pivot apne parent pivot ke relative rotate hota hai.
- Pivot drag karte waqt `bounding box` update hota hai real-time.

### 🔹 Constraint System
- Horizontal/Vertical/Circle/Polygon modes.
- Math: `x = clamp(x, min, max)` ya `distance(center) ≤ radius`.
- Visual feedback: Agar limit cross kare, object `snap back` + `constraint line glow`.

### 🔹 Keyframe vs Frame-by-Frame
- **Switch:** Top bar me toggle `🖼️ FbF / 💎 Keyframe`.
- **FbF:** Har frame me full object state save.
- **Keyframe:** Sirf changed properties save (`x, y, rotation, scaleX, scaleY, opacity, color`). Baki frames `linear/cubic bezier interpolation` se auto-fill.
- **Interpolation:** User right-click handle → `Ease In, Ease Out, Linear, Bounce`.

---

## 💡 5. 12 ADVANCED LOGIC-BASED FEATURES (Offline, No Internet)

| Feature | Logic | Workflow |
|---|---|---|
| **1. Auto Inbetween Generator** | Do keyframes ke beech `t` value (0→1) se position/rotation interpolate karo. | Frame 1 & 10 select → `Generate 8 Frames` → Auto tween. |
| **2. Motion Trail** | Last 10 positions store → fading dots render. | Real-time drag karte waqt trail dikhega. |
| **3. Smart Walk Cycle Builder** | Pre-rigged template (Torso+4 limbs) → auto keyframes for contact/passing/breakdown/up. | Tap `Walk` → 4 frames auto → user tweak. |
| **4. Pose Library** | Object state JSON save → folder structure. | `Idle` save → later 1 tap apply. |
| **5. Symmetry Animation** | X/Y axis mirror → duplicate stroke + invert transform. | Left arm move → Right auto sync. |
| **6. Ghost Frame Direct Edit** | Onion ghost ko tap → current frame auto create + ghost position copy. | Onion skin par drag karo → current frame update. |
| **7. Constraint Zones** | Circle/Polygon bounds → `point-in-polygon` test. | Character ko floor zone me lock. |
| **8. Timeline Heatmap** | Frame-to-frame pixel/object change % calculate → color bar. | Red = heavy change, Blue = static. |
| **9. Reusable Asset Browser** | IndexedDB me JSON objects store. Drag-drop across projects. | Hand, Eye, Prop library. |
| **10. Live Pivot Recording** | Pivot drag track karo → auto keyframes for rotation center. | Pivot animate karo → joint motion smooth. |
| **11. Smart Duplicate with Offset** | Copy + auto shift `x+=20, y+=10, rot+=15` per frame. | Trail/echo effect in 2 taps. |
| **12. Animation Analyzer** | Frame delta velocity check → jerky motion flag + suggest easing. | `⚠ Frame 4 sudden jump` → `Smooth?` → Auto fix. |

---
## 🎬 6. COMPLETE WORKFLOW EXAMPLE: WALK CYCLE
1. **Layer 2** active → `BRS` se Head, Torso, L-Arm, R-Arm, L-Leg, R-Leg draw.
2. `SEL` → Torso tap → `GRP` → `Build Hierarchy` → Torso parent, baki child.
3. `PVT` → Hip, Knee, Shoulder, Elbow par red pivots drop.
4. `CON` → Feet lines add. Character floor ke upar hi rahega.
5. **Frame 1:** Contact pose. `L-Leg` select → rotate forward.
6. Tap `[+]` → **Frame 2:** `R-Leg` select → rotate back. `ONION` ON → previous ghost visible.
7. **Frame 3:** Opposite of Frame 1.
8. **Frame 4:** Opposite of Frame 2.
9. `▸ Play 12 FPS Loop` → Smooth walk.
10. `Auto Inbetween` → Frame 1 & 2 ke beech 3 frames auto.
11. `Export MP4 1080p` → Render complete.

---
## 💻 7. TECHNICAL IMPLEMENTATION BLUEPRINT

| Component | Tech/Logic |
|---|---|
| **Canvas Rendering** | WebGL (PixiJS) ya Canvas2D with `requestAnimationFrame`. Object pooling. |
| **Hit Detection** | Spatial Hashing + `point-to-segment` + `point-in-polygon`. Touch radius = 12px. |
| **State Management** | Redux/Zustand ya custom event bus. Frame state = `Delta Object`. |
| **Storage** | IndexedDB (large JSON/projects). Auto-save every 5s. |
| **Export** | `FFmpeg.wasm` (MP4/GIF offline) ya `canvas.captureStream() + MediaRecorder`. |
| **Performance** | - Dirty rectangle rendering<br>- Object ID reuse<br>- Keyframe delta compression<br>- Web Workers for onion skin render |
| **Math** | `gl-matrix` for transforms. `bezier-js` for interpolation. `earcut` for shape triangulation. |

---
## ✅ FINAL CHECKLIST (Tumhare Requirements vs Implementation)
| Requirement | Status | How it's solved |
|---|---|---|
| Tap to select, exact line hit | ✅ | Spatial hash + polyline distance |
| Persistent across frames | ✅ | ID-based state, not canvas snapshot |
| Single/Multiple Pivots | ✅ | Red dots, drag, joint chain math |
| Parent-Child Unlimited Nesting | ✅ | Transform inheritance matrix |
| Constraint Lines (Top/Bottom) | ✅ | Clamp logic per object |
| Frame-by-Frame + Keyframe | ✅ | Toggle + delta storage + interpolation |
| Onion Skinning | ✅ | Ghost render with tint/opacity |
| Color Picker + Hex + Opacity | ✅ | HSL wheel, HEX input, real-time fill |
| Light Theme + White Canvas | ✅ | CSS variables + pure white backdrop |
| Export MP4/GIF Offline | ✅ | FFmpeg.wasm + MediaRecorder fallback |
| 12 Advanced Logic Features | ✅ | All offline, math/UI driven |
| Copy-Paste retains selection | ✅ | Object ID preserved, not rasterized |
| Bounding Box + Rotate/Scale | ✅ | 8-handle + top rotation handle |
| Layers + Lock/Hide/Opacity | ✅ | Render pipeline skip logic |

---
Agar tum isko banane lagte ho, to **Phase 1** me sirf: `Canvas + Stroke Save + SEL + Bounding Box + Frame Timeline + Onion Skin` implement karo.
**Phase 2**: `PVT + GRP + CON + Keyframe Toggle`
**Phase 3**: `Export + Advanced Tools + Optimization`

Har feature ka exact math, touch radius, aur state flow upar define kiya gaya hai. Kisi ek module (jaise Pivot Math, Keyframe Interpolation, ya FFmpeg Export) ka full code structure chahiye ho to batao, main uska complete implementation logic + pseudo-code de dunga.

Tumhara app primarily **client-side/offline** chalta hai, isliye "backend logic" yahan se matlab hai **Core Engine Architecture, Data Flow, State Management, Rendering Pipeline,

aur Local Persistence Logic**. Main ise ek production-ready animation engine ki tarah design kar raha hoon. Har module ka detailed logic, data structure, aur execution flow neeche diya gaya hai.

---
## 🏗️ 1. CORE DATA MODEL (Normalized Object Graph)
App ki jaan ye data structure hai. Yahi `persistent selection`, `parent-child`, `pivots`, aur `keyframe vs FBF` ko possible banata hai.

```json
{
  "project": {
    "id": "proj_abc123",
    "meta": { "width": 1920, "height": 1080, "fps": 12, "mode": "fbf | keyframe" },
    "objects": {
      "obj_01": {
        "id": "obj_01", "name": "Head", "type": "brush", "layerId": "layer_char",
        "parentId": null, "childrenIds": [],
        "baseGeometry": { "points": [[x,y]...], "strokeWidth": 3 },
        "defaultStyle": { "fill": "#00000000", "stroke": "#FF5252", "opacity": 1 },
        "pivots": [{ "id": "p1", "localX": 0, "localY": 0 }],
        "constraints": [{ "type": "horizontal", "min": 100, "max": 600 }],
        "isLocked": false, "isHidden": false
      },
      "obj_02": { "id": "obj_02", "name": "Torso", "parentId": null, "childrenIds": ["obj_01"] }
    },
    "frames": {
      "0": { "objects": { "obj_01": { "transform": { "x": 100, "y": 200, "rotation": 0, "scaleX": 1,
"scaleY": 1 }, "stroke": "#E91E63" } } },
      "1": { "objects": { "obj_01": { "transform": { "x": 120, "y": 190, "rotation": 15 } } } }
    },
    "assets": {}, "undoStack": [], "redoStack": []
  }
}
```
**Key Logic:**
- `baseGeometry` sirf ek baar store hota hai (draw once).
- `frames` sirf **deltas** store karta hai (`transform`, `color`, `opacity`, `pivot overrides`).
- `parentId` flat tree banata hai. `childrenIds` traversal ko fast banata hai.
- Jab frame switch hota hai, engine `baseGeometry + frameDelta` merge karta hai → real-time state generate karta hai. Isliye **selection har frame me kaam karta hai**.

---
## 🧠 2. STATE MANAGEMENT & HISTORY ENGINE
Har action ko track, undo/redo, aur real-time sync karna padega.

### 📦 State Store (Reactive)
```js
```

```
class AnimationEngine {
  state = { currentFrame: 0, selectedIds: [], tool: 'SEL', onion: { enabled: true, prev: 1, next: 0
} }
  history = { undo: [], redo: [], max: 1000 }

  dispatch(action) {
    // 1. Validate
    // 2. Generate snapshot of affected objects BEFORE
    // 3. Apply mutation to state
    // 4. Generate snapshot AFTER
    // 5. Push { type, before, after, execute(), undo() } to history
    // 6. Notify UI listeners
    // 7. Debounced auto-save to IndexedDB
  }
}
```
**Undo/Redo Logic:**
- Stack me sirf `Action` objects store hote hain, pura JSON nahi.
- `undo()` → `redoStack.push(current)`, `state = action.before`, `render()`
- `redo()` → `undoStack.push(current)`, `state = action.after`, `render()`
- Memory limit: 1000 actions. Uske baad purane actions `IndexedDB` me compress ho jate hain.

**Frame Switch Logic:**
1. User clicks frame `[3]`
2. Engine clears `renderCache`
3. Iterates `project.frames[0..3]` → merges deltas into `activeState`
4. Rebuilds spatial hash grid for frame 3
5. Triggers `selection.update()` → UI handles show/hide accordingly

---
## 👆 3. HIT DETECTION & SELECTION LOGIC (Touch-Optimized)
"Tap to select exact line, not white space" ke liye ye algorithm use hoga:

### 🔍 Spatial Hash Grid + Polyline Distance
```js
const CELL_SIZE = 64;
const TOUCH_RADIUS = 12; // px

function hitTest(canvasX, canvasY) {
  // 1. Map screen → canvas coords (zoom/pan handle karna)
  const cellX = Math.floor(canvasX / CELL_SIZE);
  const cellY = Math.floor(canvasY / CELL_SIZE);

  // 2. Query overlapping cells
  const candidates = grid.get(cellX, cellY);

  let closestObj = null;
```

```
  let minDist = Infinity;

  for (let obj of candidates) {
    if (obj.isLocked || obj.isHidden) continue;

    // Brush/Stroke hit test
    const dist = pointToPolylineDistance(canvasX, canvasY, obj.baseGeometry.points);
    if (dist < TOUCH_RADIUS && dist < minDist) {
      minDist = dist;
      closestObj = obj;
    }
  }
  return closestObj;
}

function pointToPolylineDistance(px, py, points) {
  let min = Infinity;
  for (let i = 0; i < points.length - 1; i++) {
    const d = pointToSegmentDistance(px, py, points[i], points[i+1]);
    if (d < min) min = d;
  }
  return min;
}
```

**Multi-Select (Lasso):**
- User draws freehand polygon.
- Engine har object ke `boundingBox` ya `points` ko `pointInPolygon` test karta hai.
- Jo intersect karte hain, unhe `selectedIds[]` me add karta hai.

**Selection Feedback:**
- `selectedIds` update hote hi → UI me bounding box, 8 handles, rotation handle render hote hain.
- Canvas par tap karte hi `pointerdown` → `hitTest()` → `dispatch({type: 'SELECT', id})`

---
## 🎨 4. RENDERING PIPELINE (Canvas/WebGL)
Har frame me kya draw hoga, order kya hoga, aur performance kaise maintain hogi:

```js
function renderFrame() {
  ctx.clearRect(0, 0, width, height);

  // 1. Onion Skin (Previous/Next frames)
  if (onion.enabled) renderOnionGhosts(ctx);

  // 2. Active Layers (z-order)
  for (let layer of layers.sortByZ()) {
    if (!layer.visible) continue;
```

```
    ctx.globalAlpha = layer.opacity;

    for (let objId of layer.objectIds) {
      const obj = project.objects[objId];
      if (obj.isHidden) continue;

      // 3. Compute Final Transform (Hierarchy + Pivots)
      const finalTransform = computeWorldTransform(objId, currentFrame);

      // 4. Apply Constraints (Clamp position before draw)
      const clampedTransform = applyConstraints(finalTransform, obj.constraints);

      // 5. Draw
      ctx.save();
      ctx.setTransform(clampedTransform);
      drawGeometry(ctx, obj.baseGeometry, obj.style);
      ctx.restore();
    }
  }

  // 6. UI Overlays (Selection, Pivots, Motion Trails)
  renderUIOverlays();
}
```
**Transform Math (Parent-Child + Pivots):**
```js
function computeWorldTransform(objId, frameIdx) {
  const obj = project.objects[objId];
  const frameState = project.frames[frameIdx]?.objects[objId] || {};

  let local = frameState.transform || {x:0,y:0,rotation:0,scaleX:1,scaleY:1};

  // Pivot override
  const pivot = obj.pivots[0] || {localX:0, localY:0};
  const pivotOffset = { x: -pivot.localX, y: -pivot.localY };

  let matrix = multiply(translate(pivotOffset), local, translate(pivotOffset.inverse));

  // Parent inheritance
  if (obj.parentId) {
    const parentMatrix = computeWorldTransform(obj.parentId, frameIdx);
    matrix = multiply(parentMatrix, matrix);
  }
  return matrix;
}
```

---

## 🔄 5. ANIMATION & INTERPOLATION ENGINE
**FBF vs Keyframe Hybrid Logic:**

| Mode | Storage | Playback |
|---|---|---|
| **Frame-by-Frame** | Har frame me full object state delta save | Direct render, no math |
| **Keyframe** | Sirf changed properties at specific frames | Interpolation fill gaps |

**Interpolation Logic:**
```js
function getValueAtFrame(objId, prop, frameIdx) {
  const keyframes = getKeyframesFor(objId, prop); // sorted by frameIndex
  if (!keyframes.length) return obj.default[prop];

  // Find surrounding keyframes
  let prev = keyframes[0], next = keyframes[keyframes.length-1];
  for (let kf of keyframes) {
    if (kf.frame <= frameIdx) prev = kf;
    if (kf.frame >= frameIdx) { next = kf; break; }
  }

  if (prev.frame === next.frame) return prev.value;

  const t = (frameIdx - prev.frame) / (next.frame - prev.frame);
  const easedT = applyEasing(t, prev.easing || 'linear');
  return prev.value + (next.value - prev.value) * easedT;
}
```
**Auto Inbetween Generator:**
- User selects Frame A & B → `generateInbetweens(A, B, count)`
- Engine har property ke liye `t = i/count` calculate karta hai
- Naye frames me deltas inject karta hai → undo stack me push → render trigger

**Motion Trail Logic:**
- `trailHistory[objId] = [{frame, transform, alpha}]`
- Render loop me last 8 positions ko fading circles/lines draw karta hai.
- Auto-cleanup jab object move karta hai ya frame change hota hai.

---
## 💾 6. LOCAL PERSISTENCE & EXPORT LOGIC
App offline chalta hai, isliye sab kuch local storage + worker-based export hoga.

### 🗄️ IndexedDB Schema
```js
// Stores
- projects: { id, jsonMeta, lastSaved, version }
- assets: { id, type: 'image|audio|font', blob }
- autosave: { projectId, timestamp, compressedDelta }
```

```
// Auto-Save Logic
let saveTimer = null;
function triggerSave() {
  if (saveTimer) clearTimeout(saveTimer);
  saveTimer = setTimeout(async () => {
    const delta = compressState(project); // zlib/gzip
    await db.put('autosave', { id: project.id, data: delta });
  }, 5000);
}
```

### 📤 Export Pipeline
| Format | Logic | Fallback |
|---|---|---|
| **GIF** | Render frames → collect `ImageData` → LZW encode (`gif.js` worker) | `gif.js` (Web Worker) |
| **MP4/WebM** | `canvas.captureStream()` → `MediaRecorder` → download blob | Browser native API |
| **PNG Seq** | Iterate frames → `canvas.toBlob()` → `JSZip` → `.zip` | `JSZip` + `FileSaver` |
| **Project** | ZIP(JSON + assets + thumbnails) → `.anim` | Custom archive |

**MP4 Export Worker Flow:**
1. Main thread → `OffscreenCanvas` create
2. Web Worker → frames render karta hai offscreen
3. `MediaRecorder` stream capture karta hai
4. Blob ready → `URL.createObjectURL()` → auto-download prompt

---
## ⚡ 7. PERFORMANCE OPTIMIZATION LOGIC
| Problem | Solution | Implementation |
|---|---|---|
| Heavy Canvas Redraw | Dirty Rectangle Tracking | `ctx.save() + clip(changedBounds) + restore()` |
| Large Project Lag | Frame Caching + Lazy Load | Sirf `current±2` frames RAM me, baki IndexedDB se stream |
| Hit Detection Slow | Spatial Hash Grid | 64px cells, dynamic rebuild on drag end |
| Undo Memory Bloat | Delta Compression + History Flush | >500 actions → compress → store in DB |
| Export Freezes UI | Web Workers + OffscreenCanvas | Render/Encode main thread se alag |
| Touch Jitter | Stabilizer + Pointer Events | `pointermove` throttled 16ms, Kalman filter for brush |

---
## 🧩 8. TOOL → ENGINE ACTION FLOW (Exact Sequence)
Har tool kaise engine se interact karega:

| Tool | Pointer Event | Engine Dispatch | State Change | UI Update |
|---|---|---|---|---|
| **SEL** | `down` → `hitTest()` → `up` | `SELECT {ids}` | `selectedIds` update | Bounding box, handles |
| **PVT** | `down` on obj → `move` → `up` | `ADD_PIVOT {objId, x, y}` | `obj.pivots.push()` | Red dot appears, transform origin shifts |
| **BRS** | `move` (stream points) → `up` | `CREATE_STROKE {points, style}` | `objects[newId]` + `frames[current][newId]` | Stroke renders, spatial grid updates |
| **GRP/PAR** | Drag in panel → drop | `SET_PARENT {childId, parentId}` | `obj.parentId = parentId`, rebuild tree | Indentation, hierarchy lock icon |
| **CON** | `tap CON` → draw lines → `up` | `ADD_CONSTRAINT {objId, type, bounds}` | `obj.constraints.push()` | Lines render, clamp enabled |
| **ONION** | Toggle button | `SET_ONION {enabled, prev, next, colors}` | `engine.state.onion` update | Ghost frames appear/disappear |
| **PLAY** | `▶` click | `SET_PLAYING true` | `playbackLoop.start()` | Timeline scrubber moves, FPS enforced |

---
## 🛠️ 9. IMPLEMENTATION CHECKLIST FOR DEVELOPERS
1. **Phase 1 (Core)**
   - IndexedDB setup + JSON schema
   - Canvas2D render loop + zoom/pan
   - Spatial hash grid + hit detection
   - Frame delta storage + undo/redo
2. **Phase 2 (Animation)**
   - Selection handles + drag/rotate/scale math
   - Pivot system + transform matrix inheritance
   - Onion skin renderer + ghost tint logic
   - FBF playback + FPS controller
3. **Phase 3 (Advanced)**
   - Parent-child drag-drop panel
   - Constraint clamping + UI lines
   - Keyframe interpolation + easing curves
   - Auto-inbetween + motion trail
4. **Phase 4 (Export & Polish)**
   - MediaRecorder + GIF.js workers
   - Auto-save debounce + project ZIP
   - Touch stabilization + dirty rect rendering
   - UI state sync + light theme assets

---
## ✅ FINAL ARCHITECTURE SUMMARY
- **Data:** Flat object graph + per-frame deltas + hierarchy links
- **State:** Action-based dispatch + undo/redo stack + reactive listeners
- **Render:** Layer → Hierarchy → Pivot → Constraint → Onion → UI overlay
- **Input:** Spatial hash → polyline distance → selection dispatch
- **Animation:** Delta merge → interpolation → playback loop → FPS sync

- **Storage:** IndexedDB + compressed autosave + Web Worker export
- **Performance:** Dirty rects, lazy frames, object pooling, touch throttle

Agar tum kisi specific module ka **full pseudo-code** chahte ho (jaise `Pivot+Hierarchy Matrix Math`, `Keyframe Interpolation with Easing`, ya `FFmpeg.wasm Offline Export Pipeline`), to batao. Main uska complete line-by-line implementation logic de dunga jo directly TypeScript/JS me convert ho sake.

Backend logic sabse important part hai, kyunki agar backend galat design hua to selection, pivot, parent-child, frame copy, onion skin sab toot jayega.

Tumhare app ke liye main "Object-Based Animation Engine" banaunga, na ki normal canvas drawing app.

---

Level 1: Project Structure

Project save file:

```
{
  projectId: "123",

  settings: {},

  scenes: [],

  assets: [],

  timeline: []
}
```

---

Level 2: Scene System

```
{
  sceneId: "scene_1",

  name: "Main Scene",

  layers: []
}
```

---

Level 3: Layers

```
{
  layerId: "layer_1",

  name: "Character Layer",

  visible: true,

  locked: false,

  opacity: 1,

  objects: []
}
```

---

Level 4: Objects

Sabse important.

Har drawing ek object hai.

```
{
  id: "obj_001",

  name: "Head",

  type: "stroke",

  path: [...],

  fillColor: "#ff0000",

  strokeColor: "#000000",

  strokeWidth: 5,

  opacity: 1,

  transform: {}
}
```

---

IMPORTANT RULE

Canvas par kuch nahi hota.

Sab objects hote hain.

Even brush drawing bhi object hai.

---

Example

User draw:

Circle

Backend:

```
{
  id:"circle_1"
}
```

User draw:

Arm

Backend:

```
{
 id:"arm_1"
}
```

User draw:

Leg

Backend:

```
{
 id:"leg_1"
}
```

---

Object Transform System

Har object:

```
{
 transform:{
  x:100,
  y:200,
  rotation:0,
  scaleX:1,
  scaleY:1
 }
}
```

Move karne par drawing change nahi hoti.

Sirf transform change hota hai.

---

Frame System

Sabse bada difference.

Frame me object copy nahi hoga.

Frame me object state save hogi.

---

Frame 1

```
{
 object:"arm_1",
 x:100,
 y:100,
 rotation:0
}
```

---

Frame 2

{
 object:"arm_1",

 x:120,

 y:95,

 rotation:25
}

---

Frame 3

{
 object:"arm_1",

 x:140,

 y:90,

 rotation:40
}

---

Object same hai.

Bas state alag.

Isliye har frame me selectable.

---

Selection Engine

User tap.

Touch Event

↓

Hit Test.

---

Hit Test

Canvas par sab objects check.

```
for(allObjects)
{
 checkHit()
}
```

---

Path Detection

isPointInsidePath()

---

Stroke Detection

distanceToStroke()

---

Agar touch:

15px radius

andar hai.

Select.

---

White canvas ignore.

---

Selection State

{
 selectedObjects:[]
}

---

Single Select

[
 head
]

---

Multi Select

[
 head,
 arm,
 leg
]

---

Bounding Box Engine

Select.

↓

Calculate.

minX
maxX

minY
maxY

---

Generate.

Rectangle

---

Handles generate.

---

Pivot System

Object:

```
{
 pivots:[]
}
```

---

Single Pivot

```
{
 x:100,
 y:100
}
```

---

Multiple Pivot

```
[
 pivot1,
 pivot2,
 pivot3
]
```

---

Rotation

rotateAroundPivot()

---

Scale

scaleAroundPivot()

---

Parent Child System

```
{
 id:"arm",

 parent:"body"
}
```

---

Tree

```
Body
 ├ Arm
 ├ Leg
 └ Head
```

---

Move Body

↓

Children auto move.

---

Math

childWorldPosition =
parentPosition +
localPosition

---

Group System

{
 groupId:"grp_1",

 members:[
  arm,
  leg,
  body
 ]
}

---

Move group.

↓

All move.

---

Select child.

↓

Only child edit.

---

Timeline Engine

timeline

contains

frames[]

---

Each frame

{

```
 frame:1,

 states:[]
}
```

---

State

```
{
 objectId:"arm",

 x:100,

 y:100,

 rotation:0
}
```

---

Onion Skin

Current:

Frame 10

---

Render:

Frame 9 opacity 20%
Frame 8 opacity 10%

---

Ahead:

Frame 11 opacity 20%

---

Special render layer.

---

Copy Paste Engine

User select.

Arm

Copy.

---

Clipboard

```
{
 objectId:"arm"
}
```

---

Paste frame.

Create new state.

---

Object same.

Selection preserved.

---

Undo Redo

Every action.

```
{
 action:"move",

 before:{},

 after:{}
```

}

---

History Stack.

---

Undo.

Pop.

---

Redo.

Push.

---

Constraint Lines

Object:

```
{
 constraints:[]
}
```

---

Upper

y = 100

---

Lower

y = 500

---

Move.

Check.

```
if(y<100)
 y=100
```

---

Object stop.

---

Motion Path

```
{
 path:[
  p1,
  p2,
  p3
 ]
}
```

---

Animation.

followPath()

---

Shape Engine

Rectangle

```
{
 width:100,
 height:100
}
```

---

Circle

{
 radius:50
}

---

Editable forever.

---

Color Engine

Object.

{
 fill:"#ff0000",

 opacity:1
}

---

Realtime update.

---

Layer Engine

Render order.

Background

Character

Effects

UI

---

Top layer draw last.

---

Export Engine

Render.

Frame1
Frame2
Frame3
...

---

Generate.

PNG Sequence

---

Convert.

MP4
GIF
WEBM

---

Performance Optimization

Important.

Don't redraw entire canvas.

Use:

Dirty Objects

Only changed objects render.

---

Spatial Index

For selection.

Use:

Quadtree

instead of checking 10,000 objects.

---

Memory Optimization

Never save:

Full Canvas Image

Save:

Objects
Transforms
States

Only.

---

Most Important Backend Rule

Tumhare app me object kabhi frame ke andar nahi rahega.

Object project level par rahega.

Frame sirf object ki state store karega.

```
Project
 ├ Objects
 ├ Layers
 ├ Groups
 └ Frames
     ├ Object State
     ├ Object State
     └ Object State
```

Isi architecture ki wajah se:

Har frame me selection kaam karega

Copy paste ke baad object selectable rahega

Pivot survive karega

Parent-child survive karega

Groups survive karenge

Onion skin fast chalegi

1000+ frames bhi manageable rahenge

Yahi architecture professional animation tools (vector animation, rigging, motion graphics systems) ke sabse kareeb hai.

Tum jo iss app ko build kar rahe ho tumhe fully matlab puri tarah freedom hai ki tum isme koi bhi features ya ui apne anusaar add kar sakte ho, jaisa tumhe achha lage. aur background aur frontend logic aur ui apne anusaar bana sakte ho, aisa koi dabaw nahi ki tum yahi bataya gaya UI aur backend banao, jaisa tumhe achha lage uss anusaar tum changes kar sakte ho. Tumhe iss app ke bare me koi bhi decision Leni ki puri tarah se freedom hai. tum chahe scratch se apne anusaar banao. Koi dikkat nahi ha.